

When a user toggles an "intelligence level" or "thinking mode" in a chat interface *without* switching to a completely different model tier (like jumping from a small model to a massive one), the platform is implicitly manipulating **inference-time compute**.

Rather than changing the underlying weights, the app dynamically changes *how long and how hard* the model is allowed to think before answering.

How It's Implemented (The Mechanics)

1. Native Reasoning Parameters (`reasoning_effort` / `budget`)

Many modern provider APIs (such as OpenAI's reasoning series, Anthropic's Claude with extended thinking, and various open-weights models) expose native parameters:

- **Token Budgeting:** The user-facing toggle maps to parameters like `reasoning_effort="low" | "medium" | "high"` or a specific `thinking: { "budget_tokens": X }` limit.
-
- **The Scratchpad:** When toggled on, the model is instructed (via system instructions or special tokens) to output a hidden chain of thought—breaking down problems, exploring alternative paths, and self-correcting—inside a designated scratchpad field before writing the final response.
-
- **The Toggle Effect:** Turning it *off* forces the model to bypass the scratchpad entirely, drastically reducing latency and cost for casual queries (like simple greetings or translations).
-

2. Prompt-Driven & Keyword Triggers (Implicit Steering)

If an interface sits on top of a standard model that doesn't have a native API toggle, developers implement it via orchestration layers:

- **System Prompt Injection:** Toggling "Deep Thinking" injects a rigorous meta-prompt behind the scenes (e.g., *"Before answering, analyze the user's request through 3 distinct perspectives, list potential edge cases, and outline your step-by-step logic inside `<think>` tags"*).
-
- **Dynamic Few-Shot Examples:** The app prepends advanced reasoning examples to the prompt context when the high-intelligence toggle is active, priming the model to output higher-fidelity logic.
-

3. Agentic Multi-Step Loops (Orchestration Frameworks)

For advanced "ultra-intelligent" toggles, the backend shifts from a single API call to a multi-step agent loop:

- **Plan-and-Execute:** A toggle might switch the app from a standard `chat.completions` call to an agentic workflow:
-

1. *Step 1:* Model writes a plan.

- 2.
3. *Step 2:* Model executes sub-tasks (often with self-critique loops where it reviews its own code or math).
- 4.
5. *Step 3:* Model synthesizes the final answer.
- 6.

Ecosystem & Projects to Check Out

If you want to see how this is built or implemented in the open-source world, look at these companies, tools, and GitHub repositories:

Companies & Platforms

- **Anthropic (Claude App / Claude Code):** Pioneers of user-toggled "Extended Thinking," allowing users to switch a model like Sonnet into a deep-deliberation mode on demand. They also support inline prompt keywords (`think hard`, `ultrathink`) to dynamically scale token budgets.
-
- **OpenAI:** Utilizes `reasoning_effort` parameters in their API to let developers scale computation dynamically per request.
-

Open-Source Projects & Repositories

- **NiloCK/autothink (GitHub):** A proof-of-concept project demonstrating how to build an automated version of this—it analyzes user query complexity on a 0–100 scale and dynamically allocates an extended thinking budget without requiring a manual toggle.
-
- **DeepSeek (DeepSeek-R1 / Open-Weights Ecosystem):** Open-source reasoning models that natively output chain-of-thought tokens inside `<think>` tags. Many open-source chat UIs (like Open WebUI or LibreChat) use these to build frontend toggles that collapse or hide the reasoning block.
-
- **LangChain / LlamaIndex:** These orchestration frameworks include native configuration fields for `reasoning_effort` and token budgeting, allowing developers to build custom user toggles into their own localized chat applications.